YASHVI MEHTA
231071079
LY COMPS
BATCH D

Cross Platform App Dev Lab 1

AIM -

To create a simple flutter application using flutter widgets

THEORY -

Flutter is an open-source UI toolkit developed by Google for building cross-platform applications from a single codebase. It enables developers to create applications for Android, iOS, web, desktop, and embedded platforms using the Dart programming language. Flutter provides fast development, expressive user interfaces, and native-like performance.

The fundamental building blocks of a Flutter application are **widgets**. A widget represents any visual element or component of the user interface, such as text, buttons, images, layouts, or input fields. Every element displayed on the screen in Flutter is a widget.

Flutter widgets are broadly classified into two types -

1. **Stateless Widgets**
    - These widgets are immutable, meaning their properties cannot change after they are created.
    - They are used for displaying static content.
    - Example: Text, Icon, Image.
2. **Stateful Widgets**
    - These widgets can change their appearance or behavior during runtime.
    - They maintain a mutable state and update the UI whenever the state changes using the setState() method.
    - Example: Counters, forms, toggles, and interactive applications.

Flutter offers a rich collection of built-in widgets to simplify application development. Some commonly used widgets include:

- **MaterialApp** – Provides Material Design styling and serves as the root widget of the application.
- **Scaffold** – Implements the basic visual structure of the application, including the AppBar, Body, Floating Action Button, Drawer, etc.
- **AppBar** – Displays the title and actions at the top of the screen.
- **Text** – Displays textual content.
- **Container** – Used to add padding, margins, borders, background colors, and sizing to widgets.
- **Row** – Arranges child widgets horizontally.
- **Column** – Arranges child widgets vertically.
- **Center** – Aligns its child widget at the center of the screen.
- **ElevatedButton** – Creates a clickable button with elevation.
- **Image** – Displays images from assets or the internet.
- **Icon** – Displays Material Design icons.

Flutter follows a **widget tree** architecture, where widgets are nested inside one another to create complex user interfaces. The UI is built by combining multiple widgets in a hierarchical structure, making the application modular, reusable, and easy to maintain.

A Flutter application begins execution from the main() function, which calls the runApp() method. This method loads the root widget and renders the application's interface on the screen.

My application -

Grade & GPA Calculator Application

1. Overview of the Application

The application is a Grade & GPA Calculator built using Flutter. Its primary purpose is to help students calculate:

SGPA (Semester Grade Point Average): By inputting credits and letter grades for individual subjects.

CGPA (Cumulative Grade Point Average): By inputting SGPA values and credit hours for multiple semesters.

It provides a dynamic, user-friendly tabbed interface using a bottom navigation bar, forms with inputs, validation checkmarks, dynamically expanding list items (adding/removing subjects or semesters), and instant recalculation.

2. Core Concepts & State Management

Flutter applications are built around the concept of Widgets, which describe what the view should look like given their current configuration and state.

A. StatelessWidget vs. StatefulWidget

StatelessWidget (e.g., MainApp): These widgets are immutable; their properties cannot change once built. They are suitable for static layouts or widgets that do not maintain internal state.

StatefulWidget (e.g., MainScreen, SgpaScreen, CgpaScreen): These widgets maintain mutable state that can change during the widget's lifetime. When the state changes, the widget is rebuilt to reflect those changes.

B. State Manipulation via setState

In SgpaScreen, adding a subject, deleting a subject, or updating credit/grade selections updates the underlying list of subjects.

Calling setState(() {}) tells the Flutter framework that the internal state has changed, prompting it to trigger the build method again and redraw the screen with the updated list and calculated SGPA.

3. Key Flutter Widgets Used in the App

The application utilizes a rich set of built-in Material design widgets, categorized as follows:

A. Structural & Layout Widgets

MaterialApp: The wrapper widget that initializes the application, sets the overall theme (ThemeData with Material 3), and sets the default route (home).

Scaffold: Implements the basic Material Design visual layout structure. It provides slots for an AppBar, body, bottomNavigationBar, and floatingActionButton.

AppBar: A top bar that contains a title and determines the navigation context style.

IndexedStack: Displays one widget from a list of children at a time, keeping the state of all other widgets intact. Used in main.dart to toggle between the SGPA and CGPA screens without losing input data when switching tabs.

Column & Row: Core layout widgets for arranging children vertically and horizontally, respectively.

Expanded: Instructs a child of a Row or Column to fill the remaining available space. It prevents overflow errors inside lists and input rows.

Card & Container: Used for styling, adding background color, padding, margins, and borders around input blocks for individual subjects.

B. Forms and User Inputs

Form: A container widget that helps group, validate, and save multiple form fields (such as text fields and dropdowns) together.

TextFormField: A text input field that integrates with Form. It supports input restriction (e.g., TextInputType.number), input decoration (labels and borders), and input validation (verifying if credits entered are valid numbers greater than 0).

DropdownButtonFormField: Combines a dropdown selection menu with form field capabilities. Used for choosing grades (e.g., O, A+, A, B, etc.) and validation checks.

IconButton: A clickable icon button used in this app to trigger the deletion of a specific subject/semester card.

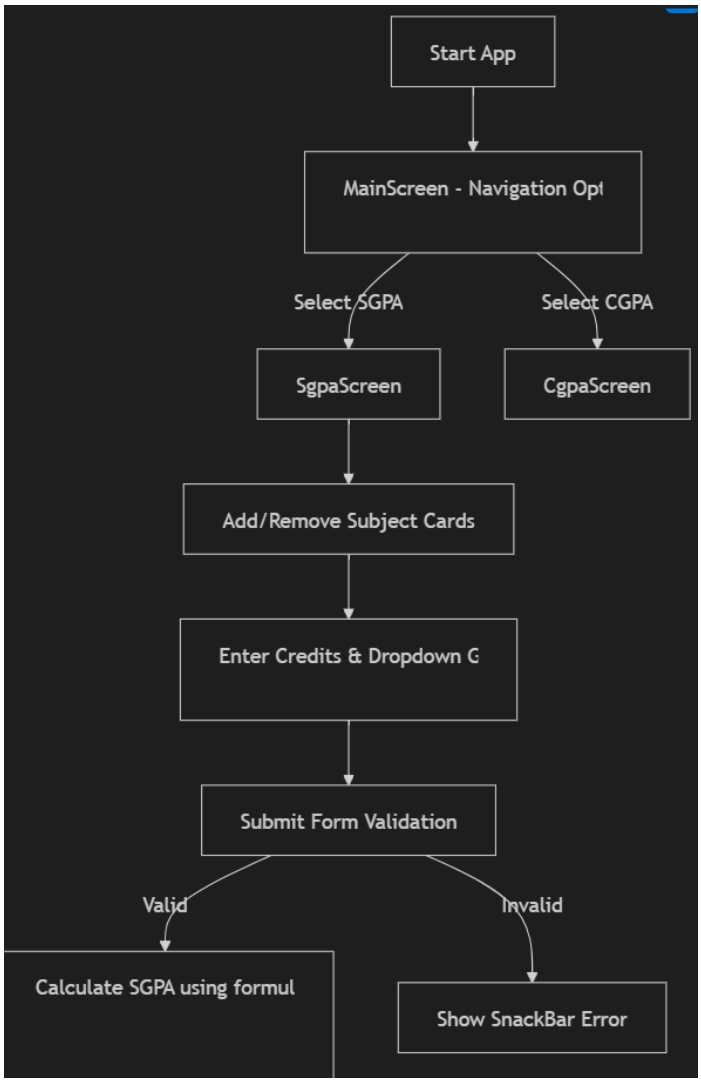
C. Scrollable and Dynamic Collections

ListView.builder: A scrollable list of widgets that are built on demand. Since the user can add an arbitrary number of subjects/semesters, ListView.builder efficiently manages memory by only rendering items currently visible on screen.

D. User Feedback

SnackBar & ScaffoldMessenger: Used to show transient visual notifications at the bottom of the screen (e.g., displaying error messages when validation fails or warning the user when trying to delete the last remaining subject card).

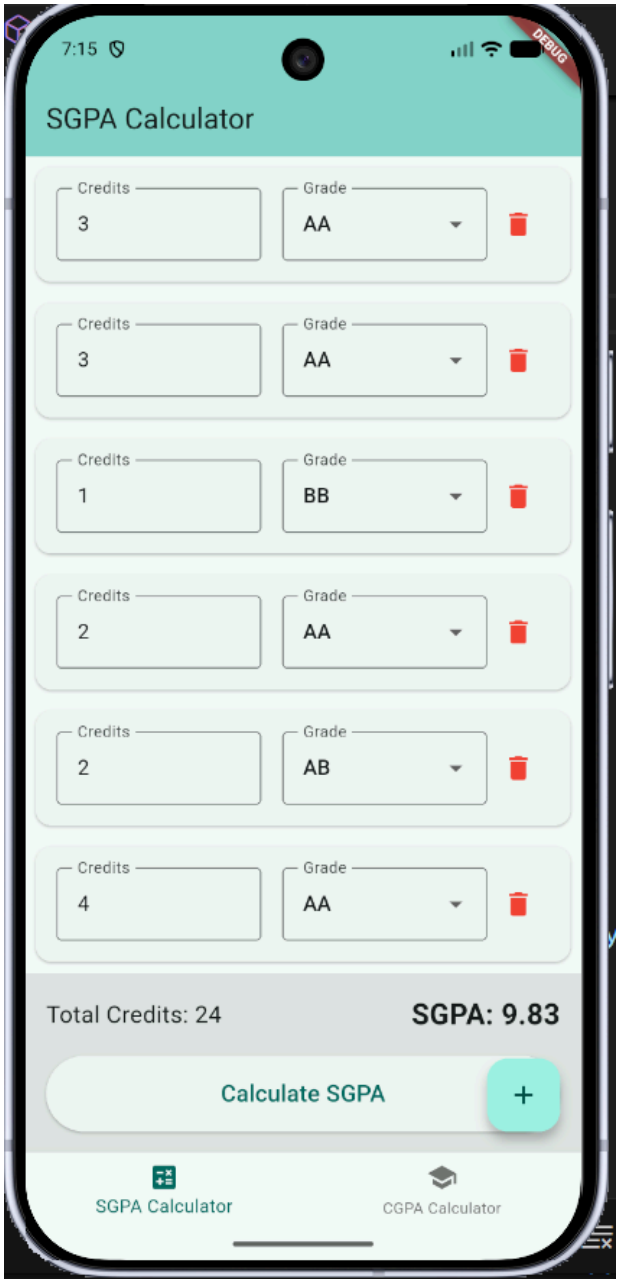
FLOW -

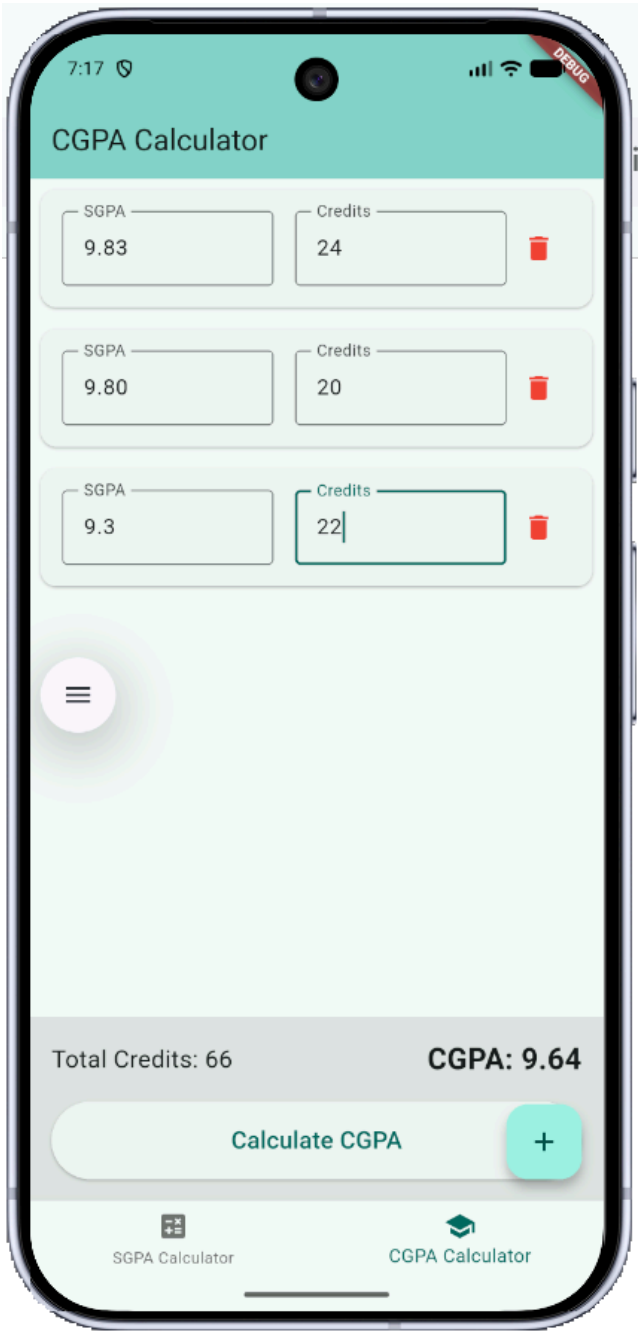


GITHUB REPO FOR CODE -

https://github.com/YashviMehta03/CPAD_lab

UI SNAPSHOTS -





CONCLUSION -

In this experiment, I successfully designed and developed a multi-functional Grade & GPA Calculator application using the Flutter framework. Through this process, I explored the core architecture of Flutter, including the distinction and implementation of StatelessWidget and StatefulWidget. I gained hands-on experience in

implementing responsive layouts using layout components (Row, Column, Expanded), building dynamically growing forms with validation capabilities (TextFormField, DropdownButtonFormField), and managing structured lists efficiently with ListView.builder. This experiment demonstrated the efficiency of Flutter’s hot-reload capability and its declarative widget-based UI model for building modern, platform-independent mobile and web applications.